

FutureKawa

Documentation IoT — ESP32 & Capteur DHT22

Équipe : Rudy · Florian · Noé · Josselin

Version : 1.0

Date : 02/07/2026

Projet : FutureKawa

Sommaire

IoT — FutureKawa

01 — Vue d'ensemble

02 — Matériel & Câblage

03 — Code Arduino

04 — Configuration

05 — Flux de données MQTT

06 — Validation des données

07 — Déploiement

08 — Tests

IoT — FutureKawa

Projet	FutureKawa
Document	Documentation IoT (ESP32 + DHT22)
Version	1.0
Date	02/07/2026
Équipe	Rudy · Florian · Noé · Josselin

Résumé

Le module IoT est un **capteur embarqué ESP32** qui mesure la température et l'humidité via un capteur DHT22 et publie les données en temps réel sur le broker MQTT. Il constitue le point d'entrée des données réelles dans le système FutureKawa.

Composant	Rôle
ESP32	Microcontrôleur WiFi — exécute le code Arduino
DHT22	Capteur numérique température + humidité
<code>code.ino</code>	Sketch principal — orchestration du cycle de mesure
<code>sensors.h</code>	Lecture et validation DHT22
<code>wifi_handler.h</code>	Connexion WiFi + synchronisation NTP
<code>mqtt_handler.h</code>	Publication MQTT (mesures + erreurs)
<code>config.h</code>	Credentials WiFi/MQTT (non versionné)

Navigation

#	Document	Contenu
1	Vue d'ensemble	Rôle, matériel, architecture
2	Matériel & câblage	ESP32, DHT22, branchements
3	Code Arduino	Structure du sketch, modules
4	Configuration	config.h, setup initial, sécurité
5	Flux de données MQTT	Topics, payloads JSON, cycle de vie
6	Validation des données	Règles de validation, gestion d'erreurs DHT22
7	Déploiement	arduino-cli, commandes make, upload
8	Tests	Tests Python, couverture, lancement

01 — Vue d'ensemble

[Suivant : Matériel & câblage →](#)

Rôle dans l'architecture FutureKawa

Le module IoT est la **source de données réelles** du système.
Contrairement au simulateur (qui génère des données fictives), l'ESP32 mesure effectivement les conditions physiques d'un entrepôt et les envoie au broker MQTT.

```
[DHT22]
|
| lecture T° + H% toutes les 5s
v
[ESP32]
|
| validation → rejet si NaN, hors bornes ou quasi-nul
| construction payload JSON
| publish MQTT
v
[Broker Mosquitto]
|
|— capteur/iot/temperature → [mqtt_subscriber.py] → PostgreSQL
|— capteur/iot/humidity    → [mqtt_subscriber.py] → PostgreSQL
|— capteur/iot/error       → alerte "Capteur défaillant"
```

Positionnement dans le projet

FutureKawa distingue deux sources de données capteur :

Source	Topics	Contexte
Simulateur (simu-sensor)	<code>capteur/simu-sensor/+</code>	Données fictives pour développement et démo

Source	Topics	Contexte
IoT réel (ESP32)	capteur/iot/+	Données physiques en production

Les deux sources sont consommées par le même `mqtt_subscriber.py`, qui les distingue via le préfixe du topic. L'API d'exploitation "Quito" (`docker-compose.exploit-1.yml`) est configurée pour recevoir les données IoT réelles.

Matériel utilisé

Composant	Modèle	Rôle
Microcontrôleur	ESP32 (Espressif)	WiFi, exécution du code, publication MQTT
Capteur	DHT22 (AM2302)	Mesure température (−40°C à +80°C) et humidité (0-100%)
Alimentation	USB 5V ou batterie 3.7V Li-Po	Alimentation de l'ensemble

Cycle de fonctionnement

Démarrage

```

|
|─ dht.begin()          Initialisation capteur DHT22
|─ setup_wifi()         Connexion WiFi + sync NTP
|─ client.setServer()   Configuration broker MQTT
|
|─ [Boucle toutes les 5 secondes]
    |
    |─ reconnect()       Reconnexion MQTT si perdue
    |─ read_dht22()      Lecture T° + H%
    |
    |─ [Données invalides]
        |─ consecutive_errors++
        |
        |─ Si >= 2 → publish_sensor_error()
    |

```

```
└─ [Données valides]
    └─ publish_temperature()
        └─ publish_humidity()
```

Identifiant capteur

Chaque ESP32 a un `sensor_id` unique défini dans `config.h`. Cet identifiant est inclus dans chaque payload MQTT et permet au subscriber de :

1. Retrouver (ou créer) l'entrée `Sensor` en base
2. Résoudre l'entrepôt associé
3. Tracer l'origine de chaque mesure

Le `sensor_id` doit correspondre au champ `name` du capteur dans la base de données exploitation.

[Suivant : Matériel & câblage →](#)

02 — Matériel & Câblage

Composants

ESP32

Microcontrôleur double cœur Xtensa LX6 avec WiFi 802.11 b/g/n et Bluetooth intégrés. C'est le cerveau du module IoT.

Caractéristique	Valeur
CPU	Dual-core Xtensa LX6, 240 MHz
Flash	4 Mo
RAM	520 Ko SRAM
WiFi	802.11 b/g/n 2.4 GHz
GPIO	34 pins
Tension	3.3V (alimentation USB 5V via régulateur)
Consommation	~240 mA (WiFi actif) / ~10 µA (deep sleep)

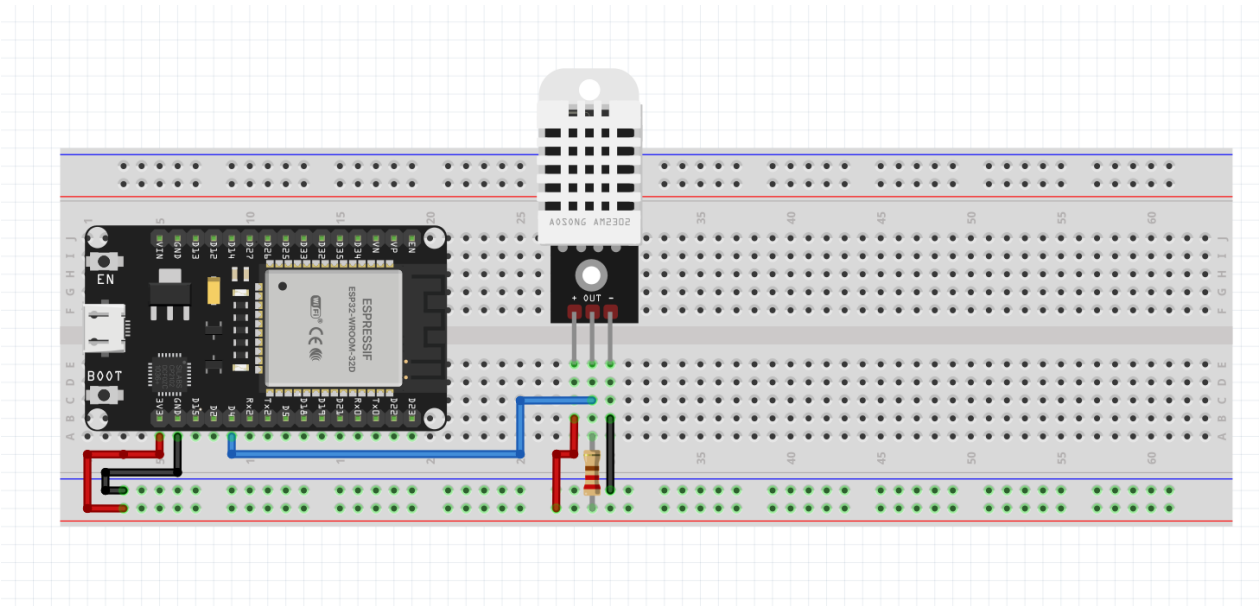
DHT22 (AM2302)

Capteur numérique de température et d'humidité relative. Protocole propriétaire 1-Wire sur un seul fil de données.

Caractéristique	Valeur
Plage température	-40°C à +80°C
Précision température	± 0.5°C
Plage humidité	0% à 100% HR
Précision humidité	± 2-5% HR
Tension	3.3V à 5.5V
Fréquence d'échantillonnage	Max 0.5 Hz (1 lecture toutes les 2s)

Caractéristique	Valeur
Protocole	1-Wire propriétaire

Câblage

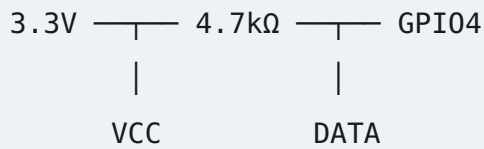


ESP32	DHT22
3.3V	VCC (1)
GPIO4	DATA (2)
	NC (3)
GND	GND (4)

Pin ESP32	Pin DHT22	Description
3.3V	VCC (1)	Alimentation
GPIO4	DATA (2)	Signal de données
—	NC (3)	Non connecté
GND	GND (4)	Masse

Résistance de pull-up

Une résistance de **4.7 kΩ à 10 kΩ** entre VCC et DATA est recommandée pour stabiliser le signal. Sur certains modules DHT22 avec breakout board, cette résistance est déjà intégrée.



Configuration du pin dans le code

Le pin de données est défini dans `config.h` :

```
#define DHTPIN 4 // GPIO4
#define DHTTYPE DHT22
```

Pour utiliser un autre GPIO, modifier uniquement `DHTPIN`. Les GPIO disponibles sur l'ESP32 pour entrée/sortie : 0, 2, 4, 5, 12-19, 21-23, 25-27, 32-33.

Alimentation

Option	Connexion	Avantage
USB	Port micro-USB de l'ESP32	Simple, alimentation fixe
Batterie Li-Po 3.7V	Pins BAT+ / BAT- (si module avec gestion batterie)	Portable
Alimentation 5V externe	Pin VIN de l'ESP32	Installation fixe

Pour une installation en production (entrepôt), une alimentation USB via adaptateur secteur est recommandée pour la fiabilité.

03 — Code Arduino

Structure des fichiers

```
iot/code/
├─ code.ino          ← Sketch principal (setup + loop)
├─ config.h          ← Credentials WiFi/MQTT (non
versionné, .gitignore)
├─ config.exemple.h  ← Template de configuration (versionné)
├─ sensors.h         ← Lecture DHT22, validation, timestamp ISO8601
├─ wifi_handler.h    ← Connexion WiFi + synchronisation NTP
└─ mqtt_handler.h    ← Publication MQTT (mesures + erreurs)
```

Le code est organisé en modules `.h` (headers) pour la lisibilité. Chaque fichier a une responsabilité unique, ce qui facilite les tests et la maintenance.

code.ino — Sketch principal

Point d'entrée Arduino. Orchestre les trois modules.

```
#include "config.h"
#include "wifi_handler.h"
#include "mqtt_handler.h"
#include "sensors.h"

WiFiClient  espClient;
PubSubClient client(espClient);
DHT         dht(DHTPIN, DHTTYPE);

void setup() {
    Serial.begin(115200);
    dht.begin();
    setup_wifi();
    client.setServer(mqtt_broker, mqtt_port);
```

```

}

void loop() {
    if (!client.connected()) reconnect();
    client.loop();

    SensorData data = read_dht22();

    if (!data.is_valid) {
        if (consecutive_sensor_errors >= MAX_CONSECUTIVE_ERRORS && !
error_alerted) {
            publish_sensor_error("DHT22_READ_FAILURE",
consecutive_sensor_errors, ...);
            error_alerted = true;
        }
        delay(2000);
        return;
    }

    publish_temperature(data.temperature, timestamp);
    publish_humidity(data.humidity, timestamp);
    delay(5000);
}

```

Intervalle de mesure : 5 secondes entre deux publications valides. En cas d'erreur, retry toutes les 2 secondes.

sensors.h — Lecture et validation

Gère la lecture physique du DHT22, la validation des données et la génération du timestamp.

Structure SensorData

```

struct SensorData {
    float temperature;
    float humidity;
}

```

```
bool is_valid;
};
```

Règles de validation

```
bool temp_valid = !isnan(data.temperature)
    && data.temperature > -20
    && data.temperature < 60;

bool humi_valid = !isnan(data.humidity)
    && data.humidity >= 0
    && data.humidity <= 100;
```

Métrique	Borne basse	Borne haute	Type de borne
Température	−20°C	60°C	Exclusive (> , <)
Humidité	0%	100%	Inclusive (>= , <=)

Les valeurs `NaN` (retournées par le DHT22 quand il ne répond pas) sont rejetées dans les deux cas.

Gestion des erreurs consécutives

```
int consecutive_sensor_errors = 0;
const int MAX_CONSECUTIVE_ERRORS = 2;
bool error_alerted = false;
```

Après **2 échecs consécutifs**, une alerte est publiée sur `capteur/iot/error`. Le flag `error_alerted` évite de spammer le topic en cas d'erreur persistante. Il est remis à `false` dès qu'une lecture valide est obtenue.

Timestamp ISO 8601

```
String get_iso8601_timestamp() {
    time_t now = time(nullptr);
    struct tm* timeinfo = gmtime(&now);
```

```

char buffer[30];
strftime(buffer, sizeof(buffer), "%Y-%m-%dT%H:%M:%SZ", timeinfo);
return String(buffer);
}

```

L'heure est synchronisée via NTP au démarrage — le timestamp est donc UTC précis dès la première mesure.

wifi_handler.h — Connexion WiFi

Gère la connexion WiFi et la synchronisation NTP.

```

void setup_wifi() {
    WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED) { delay(500); }

    // Synchronisation NTP
    configTime(0, 0, "pool.ntp.org", "time.nist.gov");
    delay(2000);
}

```

- Deux serveurs NTP en fallback (`pool.ntp.org` et `time.nist.gov`)
- UTC (offset 0, pas de DST) — la conversion en heure locale est faite côté serveur
- Délai de 2 secondes pour laisser le temps à NTP de synchroniser avant la première mesure

mqtt_handler.h — Publication MQTT

Gère la reconnexion MQTT et la publication des payloads.

Reconnexion automatique

```

void reconnect() {
    while (!client.connected()) {
        if (client.connect("esp32_client", mqtt_user, mqtt_password))
        {
            // connecté
        } else {

```

```

        delay(2000); // retry toutes les 2s
    }
}
}

```

Le client ID `esp32_client` est fixe — si plusieurs ESP32 tournent simultanément, il faut les différencier (ex: `esp32_capteur_A`). Actuellement le projet utilise un seul capteur IoT par stack.

Publication température et humidité

```

void publish_temperature(float temp, const char* timestamp) {
    StaticJsonDocument<256> payload;
    payload["metric"]    = "temperature";
    payload["value"]     = temp;
    payload["timestamp"] = timestamp;
    payload["sensor_id"] = sensor_id;
    // → client.publish("capteur/iot/temperature", json)
}

```

Les deux fonctions `publish_temperature` et `publish_humidity` sont symétriques — seuls le champ `metric` et le topic diffèrent.

Publication d'erreur

```

void publish_sensor_error(const char* error_type, int
consecutive_errors, const char* timestamp) {
    payload["error_type"]      = error_type;
    payload["consecutive_errors"] = consecutive_errors;
    payload["message"]         = "Capteur DHT22 ne répond pas";
    payload["sensor_id"]       = sensor_id;
    // → client.publish("capteur/iot/error", json)
}

```

Dépendances Arduino

Bibliothèque	Auteur	Usage
DHT sensor library	Adafruit	Lecture DHT22

Bibliothèque	Auteur	Usage
PubSubClient	Nick O'Leary	Client MQTT
ArduinoJson	Benoit Blanchon	Sérialisation JSON
WiFi	Espressif	Intégrée ESP32

04 — Configuration

config.h — Fichier de configuration

`config.h` contient tous les paramètres spécifiques à l'installation : credentials WiFi, adresse du broker, identifiant du capteur. Il est **exclu du dépôt git** (`.gitignore`) pour ne jamais exposer de secrets.

Setup initial

```
# Depuis le répertoire iot/code/  
cp config.exemple.h config.h  
# Puis éditer config.h avec les vraies valeurs
```

Contenu complet de config.exemple.h

```
#ifndef CONFIG_H  
#define CONFIG_H  
  
// WiFi  
const char* ssid      = "VOTRE_SSID";  
const char* password = "VOTRE_PASSWORD";  
  
// MQTT  
const char* mqtt_broker = "IP";    // IP du broker (ex:  
192.168.1.42)  
const int   mqtt_port   = 1883;  
const char* mqtt_user   = "adminFutureKawa";  
const char* mqtt_password = "<mot_de_passe_mqtt>";  
  
// DHT22  
#define DHTPIN  4  
#define DHTTYPE DHT22  
  
// Identifiant capteur  
const char* sensor_id = "capteur-iot-1";
```

```
#endif
```

Paramètres détaillés

WiFi

Paramètre	Description
<code>ssid</code>	Nom du réseau WiFi 2.4 GHz
<code>password</code>	Mot de passe WiFi

L'ESP32 ne supporte que le WiFi **2.4 GHz** — les réseaux 5 GHz ne sont pas visibles.

Broker MQTT

Paramètre	Description	Valeur typique
<code>mqtt_broker</code>	Adresse IP ou hostname du broker	<code>192.168.1.42</code> (réseau local)
<code>mqtt_port</code>	Port MQTT	<code>1883</code> (sans TLS) / <code>8883</code> (TLS)
<code>mqtt_user</code>	Utilisateur MQTT	<code>adminFutureKawa</code>
<code>mqtt_password</code>	Mot de passe MQTT	Défini dans <code>.env</code> du broker

En production, utiliser le port `8883` (TLS) et fournir le certificat CA au client (voir section TLS dans la doc broker).

Pin DHT22

Paramètre	Valeur par défaut	Description
<code>DHTPIN</code>	<code>4</code>	GPIO de l'ESP32 connecté au DATA du DHT22
<code>DHTTYPE</code>	<code>DHT22</code>	Type de capteur

Pour brancher sur un autre GPIO, modifier uniquement `DHTPIN`.

Identifiant capteur

```
const char* sensor_id = "capteur-iot-1";
```

Cet identifiant est **critique** : il doit correspondre exactement au champ `name` de l'entrée `Sensor` dans la base de données exploitation. Le subscriber MQTT crée automatiquement le capteur en base si `sensor_id` est inconnu (auto-découverte).

Conventions de nommage recommandées : - `capteur-iot-1` — premier capteur IoT réel - `capteur-zone-A` — capteur de la zone A - `capteur-entrepot-B-1` — premier capteur de l'entrepôt B

Sécurité

Risque	Mesure
<code>config.h</code> commité par erreur	Fichier dans <code>.gitignore</code> , ne jamais utiliser <code>git add .</code>
Credentials WiFi exposés	<code>config.h</code> jamais dans le dépôt
Mot de passe MQTT en clair	Acceptable sur réseau local ; utiliser TLS en production
Sniffing réseau	Activer TLS sur port 8883 en production

Vérification que config.h n'est pas versionné

```
git status iot/code/config.h
# Doit afficher : nothing to commit (fichier ignoré)

git ls-files iot/code/config.h
# Ne doit rien afficher
```

05 — Flux de données MQTT

Topics publiés par l'ESP32

Topic	Fréquence	Déclencheur
capteur/iot/temperature	Toutes les 5s	Lecture valide
capteur/iot/humidity	Toutes les 5s	Lecture valide
capteur/iot/error	À la demande	2 erreurs consécutives

Payload — Mesure de température

```
{
  "metric": "temperature",
  "value": 22.4,
  "timestamp": "2026-07-02T14:32:10Z",
  "sensor_id": "capteur-iot-1"
}
```

Payload — Mesure d'humidité

```
{
  "metric": "humidity",
  "value": 61.2,
  "timestamp": "2026-07-02T14:32:10Z",
  "sensor_id": "capteur-iot-1"
}
```

Payload — Erreur capteur

```
{
  "error_type": "DHT22_READ_FAILURE",
}
```

```

"consecutive_errors": 2,
"timestamp": "2026-07-02T14:35:00Z",
"sensor_id": "capteur-iot-1",
"message": "Capteur DHT22 ne répond pas"
}

```

Description des champs

Champ	Type	Présent dans	Description
<code>metric</code>	string	mesures	"temperature" ou "humidity"
<code>value</code>	float	mesures	Valeur numérique (°C ou %)
<code>timestamp</code>	string ISO 8601	tous	Heure UTC au moment de la lecture (YYYY-MM-DDTHH:MM:SSZ)
<code>sensor_id</code>	string	tous	Identifiant unique du capteur (correspond à <code>Sensor.name</code> en BDD)
<code>error_type</code>	string	erreur	Type d'erreur (<code>DHT22_READ_FAILURE</code> , ...)
<code>consecutive_errors</code>	int	erreur	Nombre d'échecs consécutifs au moment de l'envoi
<code>message</code>	string	erreur	Description lisible

Séquence de publication

```

t=0s  read_dht22() → T=22.4°C, H=61.2% (valide)
      publish capteur/iot/temperature {"value": 22.4, ...}

```

```

publish capteur/iot/humidity      {"value": 61.2, ...}

t=5s  read_dht22() → T=22.6°C, H=61.0% (valide)
publish capteur/iot/temperature {"value": 22.6, ...}
publish capteur/iot/humidity     {"value": 61.0, ...}

t=10s  read_dht22() → NaN / hors bornes (erreur #1)
pas de publication, retry dans 2s

t=12s  read_dht22() → NaN (erreur #2 = seuil)
publish capteur/iot/error {"error_type":
"DHT22_READ_FAILURE", "consecutive_errors": 2, ...}
error_alerted = true (pas de nouveau message d'erreur tant
que non résolu)

t=14s  read_dht22() → T=22.5°C, H=61.1% (récupéré)
consecutive_errors = 0, error_alerted = false
publication normale reprend

```

Côté subscriber (mqtt_subscriber.py)

À la réception d'un message sur `capteur/#`, le subscriber :

1. **Détecte** si le payload contient `error_type` → appelle `handle_sensor_error()`
2. **Sinon** → extrait `metric`, `value`, `timestamp`, `sensor_id`
3. **Met en buffer** pendant une fenêtre de 2 secondes pour grouper T° + H%
4. **Sauvegarde** dès que les deux métriques sont disponibles (`Measure` en BDD)
5. **Vérifie les seuils** → crée des alertes si dépassement

Buffering T° + H%

L'ESP32 publie température et humidité **séparément** (deux messages distincts). Le subscriber les groupe avant de créer une entrée `Measure` :

```
capteur/iot/temperature → buffer["capteur/iot"]  
capteur/iot/humidity    → buffer["capteur/iot"] → buffer complet →  
save()
```

Si une seule métrique arrive dans la fenêtre de 2 secondes, la mesure incomplète est ignorée.

06 — Validation des données

Deux niveaux de validation

Les données capteur sont validées à **deux endroits** dans le système :

Niveau	Où	Ce qui est rejeté
Embarqué	<code>sensors.h</code> (ESP32)	NaN, hors bornes physiques
Subscriber	<code>mqtt_subscriber.py</code>	Valeurs quasi-nulles (DHT22 défaillant)

Validation embarquée (`sensors.h`)

Règles

```
// Température : bornes EXCLUSIVES
bool temp_valid = !isnan(data.temperature)
    && data.temperature > -20.0
    && data.temperature < 60.0;

// Humidité : bornes INCLUSIVES
bool humi_valid = !isnan(data.humidity)
    && data.humidity >= 0.0
    && data.humidity <= 100.0;
```

Tableau des bornes

Métrique	Valeur min	Valeur max	Borne min	Borne max
Température (°C)	-20	60	Exclusive (>)	Exclusive (<)
Humidité (%)	0	100	Inclusive (>=)	Inclusive (<=)

Exemples de cas limites

Valeur	Acceptée	Raison
T = 24.7°C	Oui	Dans les bornes
T = -20.0°C	Non	Borne exclusive : doit être > -20
T = -19.9°C	Oui	Juste au-dessus de la borne basse
T = 60.0°C	Non	Borne exclusive : doit être < 60
T = 59.9°C	Oui	Juste en dessous de la borne haute
T = NaN	Non	DHT22 n'a pas répondu
H = 0.0%	Oui	Borne inclusive
H = 100.0%	Oui	Borne inclusive
H = -1.0%	Non	Sous la borne basse
H = NaN	Non	DHT22 n'a pas répondu

Justification des bornes

- **-20°C / 60°C** : correspondent aux conditions extrêmes d'un entrepôt de café. En dessous de -20°C ou au-dessus de 60°C, la lecture est vraisemblablement une erreur de capteur plutôt qu'une mesure réelle.
- **Bornes exclusives pour T°** : correspond à la logique C (`>`, `<`) du code embarqué.
- **Bornes inclusives pour H%** : 0% et 100% sont des valeurs physiquement possibles et valides.

Validation côté subscriber (mqtt_subscriber.py)

Le subscriber ajoute un rejet supplémentaire des **valeurs quasi-nulles** :

```
if value == 0 or (isinstance(value, (int, float)) and -0.1 < value < 0.1):
```

```
logger.warning(f'Valeur suspecte (quasi-nulle) reçue : {value}')  
return
```

Le DHT22 retourne parfois `0.0` comme valeur par défaut quand il est défaillant mais ne retourne pas `NaN`. Cette validation côté subscriber complète la validation embarquée.

Gestion des erreurs consécutives

Seuil d'alerte

```
const int MAX_CONSECUTIVE_ERRORS = 2;
```

Après **2 lectures invalides consécutives**, l'ESP32 publie une alerte sur `capteur/iot/error`. Ce seuil bas (2 tentatives) permet de détecter rapidement une défaillance sans surcharger le broker d'alertes.

Anti-spam

```
bool error_alerted = false;  
// ...  
if (consecutive_sensor_errors >= MAX_CONSECUTIVE_ERRORS && !  
    error_alerted) {  
    publish_sensor_error(...);  
    error_alerted = true; // une seule alerte par épisode d'erreur  
}
```

Une seule alerte est envoyée par épisode de panne. Dès qu'une lecture valide est obtenue, `error_alerted` est remis à `false` et le compteur est réinitialisé.

Côté subscriber — marquage capteur inactif

À la réception d'un message `capteur/iot/error`, le subscriber :

1. Marque le capteur `is_active = False` en base 2.
2. Crée une alerte de type `Capteur défaillant`.
3. Si une mesure valide arrive ensuite, le capteur est remis `is_active = True` automatiquement.

07 — Déploiement

Prérequis

Outil	Installation
<code>arduino-cli</code>	<code>sudo apt install arduino-cli</code> (ou via <code>make iot-install-deps</code>)
<code>screen</code> ou <code>picocom</code>	<code>sudo apt install screen</code> (monitoring série)
Core ESP32	Installé via <code>arduino-cli core install</code> (ou <code>make iot-setup</code>)

Commandes make

Commande	Description
<code>make iot-all</code>	Tout en une commande — install deps + setup + build + upload
<code>make iot-dev</code>	Compile et upload (port auto-déecté)
<code>make iot-build</code>	Compile uniquement (sans upload)
<code>make iot-upload</code>	Upload uniquement (pas de recompilation)
<code>make iot-monitor</code>	Logs série en direct (115200 baud)
<code>make iot-install-deps</code>	Installe arduino-cli, screen, picocom
<code>make iot-setup</code>	Configure arduino-cli + installe core ESP32
<code>make iot-find-port</code>	Liste les ports série disponibles
<code>make iot-clean</code>	Nettoie les fichiers compilés
<code>make iot-help</code>	Aide détaillée

Première installation complète

```
# 1. Depuis la racine du projet
make iot-all

# 2. Surveiller les logs
make iot-monitor
```

`make iot-all` exécute dans l'ordre : install deps → setup → build → upload.

Paramètres optionnels

IOT_REQUIRED — Gérer l'absence d'ESP32

```
# Mode optionnel (défaut) – pas d'erreur si ESP32 absent
make iot-build

# Mode requis – erreur si ESP32 absent
make iot-build IOT_REQUIRED=1
```

`IOT_REQUIRED=0` est utile en développement pour compiler sans avoir l'ESP32 branché. `IOT_REQUIRED=1` est pour le CI ou les tests qui nécessitent l'upload réel.

PORT — Forcer un port série

```
# Auto-détection (défaut)
make iot-dev

# Port forcé
make iot-dev PORT=/dev/ttyACM0
```

Détection automatique du port

Le script `iot/scripts/find_port.sh` cherche automatiquement les ports `ttyUSB*` et `ttyACM*`. Si plusieurs ports sont détectés, une sélection interactive est proposée.

```
# Lister les ports disponibles
make iot-find-port

# Exemple de sortie
[FIND-PORT] Recherche de l'ESP32...
✓ Port détecté: /dev/ttyUSB0
```

Procédure d'upload détaillée

1. Connexion USB de l'ESP32
2. Vérification du port

```
$ make iot-find-port
✓ Port détecté: /dev/ttyUSB0
```
3. Configuration (si première fois)

```
$ cp iot/code/config.exemple.h iot/code/config.h
# Éditer config.h avec les vraies valeurs
```
4. Build + Upload

```
$ make iot-dev
```
5. Vérification

```
$ make iot-monitor
# Attendu :
# WiFi connecté !
# Synchronisation NTP...
# [SETUP] ESP32 prêt !
# Envoi MQTT Température : {"metric":"temperature","value":
22.4,...}
# Envoi MQTT Humidité : {"metric":"humidity","value":61.2,...}
```

Permissions port série (Linux)

Sur Linux, le port `/dev/ttyUSB0` ou `/dev/ttyACM0` peut nécessiter des permissions :

```
# Ajouter l'utilisateur au groupe dialout
sudo usermod -a -G dialout $USER

# Déconnecter/reconnecter la session, puis vérifier
ls -la /dev/ttyUSB0

# crw-rw---- 1 root dialout 188, 0 ...
```

Mise à jour du firmware

Pour mettre à jour le code sur un ESP32 déjà déployé :

```
# 1. Modifier le code dans iot/code/
# 2. Recompiler et uploader
make iot-dev

# Si l'ESP32 est sur un port spécifique
make iot-dev PORT=/dev/ttyUSB0
```

L'upload Arduino efface la flash et réécrit le firmware complet. Les credentials WiFi/MQTT dans `config.h` sont réincorporés à chaque compilation.

08 — Tests

Organisation

Les tests IoT sont des tests **Python unitaires** qui reproduisent la logique du code C/Arduino sans avoir besoin d'un ESP32 physique.

```
iot/tests/
├─ test_sensor_validation.py ← Règles de validation DHT22
(sensors.h)
├─ test_mqtt_payload.py      ← Format et structure des payloads
MQTT
├─ test_sensor_measurements.py ← Cas de mesures réalistes
└─ test_iot_scripts.py      ← Scripts shell (find_port, upload)
```

Lancement

```
# Tous les tests IoT
python -m pytest iot/tests/ -v

# Un fichier spécifique
python -m pytest iot/tests/test_sensor_validation.py -v
```

test_sensor_validation.py

Reproduit en Python la logique de `sensors.h` pour valider les règles de validation sans matériel.

Couverture

Classe	Cas testés
<code>TemperatureValidationTests</code>	Valeur nominale, zéro, négatif dans bornes, sous -20 , à -20 (exclusif), juste au-dessus, au-dessus de 60 , à 60 (exclusif), juste en-dessous, NaN

Classe	Cas testés
HumidityValidationTests	Valeur nominale, 0% (inclusif), 100% (inclusif), sous 0%, au-dessus de 100%, NaN
CombinedReadingTests	Les deux valides, T° invalide, H% invalide, les deux invalides, NaN T°, NaN H%

Cas limites clés

```
# Borne exclusive température
def test_temperature_at_minus_20_rejected(self):
    self.assertFalse(is_valid_temperature(-20.0))    # > -20 strict

def test_temperature_at_60_rejected(self):
    self.assertFalse(is_valid_temperature(60.0))    # < 60 strict

# Borne inclusive humidité
def test_humidity_at_zero_accepted(self):
    self.assertTrue(is_valid_humidity(0.0))         # >= 0

def test_humidity_at_100_accepted(self):
    self.assertTrue(is_valid_humidity(100.0))       # <= 100
```

test_mqtt_payload.py

Vérifie que les payloads JSON construits par l'ESP32 (reproduits en Python) respectent le contrat attendu par le subscriber.

Couverture

Classe	Cas testés
MqttTopicTests	Format des 3 topics, préfixe commun capteur/iot/
MqttPayloadTemperatureTests	Champ <code>metric</code> , champs obligatoires, type numérique de <code>value</code> , sérialisabilité JSON, format ISO 8601 du timestamp

Classe	Cas testés
MqttPayloadHumidityTests	Champ <code>metric</code> , champs obligatoires, type numérique
MqttPayloadErrorTests	Champs obligatoires, message DHT22, type entier de <code>consecutive_errors</code>

Vérification du format timestamp

```
def test_timestamp_follows_iso8601(self):
    import re
    self.assertRegex(
        self.payload["timestamp"],
        r"^\d{4}-\d{2}-\d{2}T\d{2}:\d{2}:\d{2}Z$"
    )
```

Résultats attendus

```
$ python -m pytest iot/tests/test_sensor_validation.py iot/tests/
test_mqtt_payload.py -v

test_sensor_validation.py::TemperatureValidationTests::test_nominal_t
emperature_accepted PASSED
test_sensor_validation.py::TemperatureValidationTests::test_temperatu
re_at_minus_20_rejected PASSED
test_sensor_validation.py::TemperatureValidationTests::test_temperatu
re_at_60_rejected PASSED
test_sensor_validation.py::TemperatureValidationTests::test_temperatu
re_nan_rejected PASSED
test_sensor_validation.py::HumidityValidationTests::test_humidity_at_
zero_accepted PASSED
test_sensor_validation.py::HumidityValidationTests::test_humidity_at_
100_accepted PASSED
...
test_mqtt_payload.py::MqttTopicTests::test_temperature_topic_format
PASSED
test_mqtt_payload.py::MqttPayloadTemperatureTests::test_payload_is_js
on_serializable PASSED
```

```
test_mqtt_payload.py::MqttPayloadTemperatureTests::test_timestamp_fol  
lows_iso8601 PASSED  
...  
  
===== 28 passed in 0.12s =====
```

Limites des tests

Aspect	Couvert	Non couvert
Règles de validation	Oui	Comportement physique du DHT22
Format des payloads	Oui	Connexion MQTT réelle
Topics MQTT	Oui	Latence réseau
Logique anti-spam	Non	Requiert un ESP32
NTP et timestamp	Partiellement (format)	Précision de l'heure

Les tests couvrent la **logique applicative**. La validation matérielle (câblage, DHT22, WiFi) nécessite un ESP32 physique et le moniteur série (`make iot-monitor`).